

# Strata — BA Guide

For business analysts

strata-kb v1.0.1 · 2026-09-19

## 1. Who this guide is for

You write requirements. Tickets, user stories, acceptance criteria — and for large features, mission plans that break them down.

Your problem is not writing. It is **grounding**: making sure that when a ticket says "the field is 4 characters, alphanumeric", that claim comes from a real document, at a known revision, and stays verifiable months later when someone asks where it came from.

Strata gives you three things for that:

1. **Search over published knowledge**, so you can find the passage instead of remembering it.
2. **Citations that pin a version**, so a ticket records exactly what the source said the day you wrote it.
3. **A Definition-of-Ready gate**, so a ticket cannot reach a developer with a broken or missing citation.

### 1.1 What a BA repo does and does not do

| Does                                | Does not                   |
|-------------------------------------|----------------------------|
| Read the hub's published knowledge  | Ingest documents           |
| Cite it, pinned to a hub commit     | Summarize anything         |
| Version tickets and missions in Git | Publish knowledge          |
| Gate them in CI                     | Push to your issue tracker |

Your repository is strictly **read-only** against the knowledge base. You cite, you never contribute. That is deliberate: it keeps requirements traceable to a knowledge version without giving the requirements process a way to change the knowledge itself.

The assistant also never opens a ticket for you. It produces Markdown; you read it, commit it, and paste it into your tracker yourself.

## 2. Setup, once

```
pip install strata-kb
kb init --kind ba
```

## 2.1 Point at the hub

```
# .kb/config.yaml
kind: ba
hub: https://github.com/acme/kb-hub.git
```

This is what `kb ticket lint` and `kb query` resolve references against.

## 2.2 Connect your assistant to the hub

`.mcp.json` (Claude Code) and `.cursor/mcp.json` (Cursor) were scaffolded already wired to the hub's search and citation tools, but through two placeholders — `${STRATA_KB_HUB_URL}` and `${STRATA_KB_HTTP_TOKEN}` — that nothing has set yet.

Run `kb mcp-setup` (in your assistant: `/kb-mcp-setup`). It asks for the hub's HTTP base URL and token — ask your hub maintainer for the token — writes both into `.env`, makes sure git ignores that file, and then verifies them against the hub, so a wrong URL and a rejected token come back as different errors:

| Variable                          | Example                                     |
|-----------------------------------|---------------------------------------------|
| <code>STRATA_KB_HUB_URL</code>    | <code>http://kb-hub.example.com:8321</code> |
| <code>STRATA_KB_HTTP_TOKEN</code> | the hub's bearer token                      |

A bare re-run of `kb mcp-setup` reads both values back out of `.env` and verifies again, so checking your connection later never means retyping the token. Prefer the hidden prompt or the `STRATA_KB_HTTP_TOKEN` environment variable over the command's `--token` flag — that flag leaves the token sitting in your shell history.

`.mcp.json` and `.cursor/mcp.json` read those two variables from your environment, not from `.env` directly, so load `.env` into your shell (`set -a; source .env; set +a`, or use `direnv`) and restart your assistant — MCP reads the environment only at startup.

## 2.3 Open the repository in your assistant

`ba-ticket-author` and `ba-mission-plan` are scaffolded as skills, commands and prompts for **Claude Code**, **GitHub Copilot** and **Cursor**. Whichever you use, the pipeline is the same.

## 2.4 Configure the CI gate

| Setting                       | Where                       | What it is                                                   |
|-------------------------------|-----------------------------|--------------------------------------------------------------|
| <code>STRATA_KB_HUB</code>    | Actions → <b>Variables</b>  | The hub the gate resolves refs against                       |
| <code>KB_HUB_TOKEN</code>     | Actions → <b>Secrets</b>    | A read token — only needed for a private hub                 |
| <code>KB_FAIL_ON_STALE</code> | <b>Variables</b> , optional | Set to any value to make an upstream amendment fail the gate |

Then, in branch protection, **require the `kb-ticket-lint` check** on the branch your tickets land on. Lint running in CI does not block a merge by itself; requiring the check is what does.

A pull request opened **from a fork** cannot read repository secrets, so on a private hub the gate fails there with a hub-unreachable message. That is the gate refusing to go green without checking, not a network fault. Merge fork contributions through a branch in this repository instead.

## 3. Writing a ticket

Invoke `/ba-ticket-author` and describe the business need. The agent runs an eight-step pipeline; your job is steps 1, 3 and 8.

|                   |                                                        |
|-------------------|--------------------------------------------------------|
| 1 Intake          | you describe the need                                  |
| 2 Parent mission  | optional – name the mission this story belongs to      |
| 3 Ground          | <code>kb_search</code> ; YOU pick which sections apply |
| 4 Draft           | story, ACs, use cases, two diagrams                    |
| 5 Pin             | <code>kb_context_new</code> embeds the pinned block    |
| 6 Lint            | <code>kb ticket lint</code> until DoR: PASS            |
| 7 Maturity review | two independent reviews, up to 3 rounds                |
| 8 Review → save   | you read it, commit it, paste it into the tracker      |

### 3.1 Intake

Describe the need in business terms: the capability, who needs it, and the value. Resist writing the solution. The agent's later steps work better from a clear problem than from a pre-decided implementation.

### 3.2 Grounding — the step that actually matters

The agent calls `kb_search` and shows you **every candidate section**, not just its favourite. Read them. Pick the ones that genuinely apply.

This is the only step where your domain judgement cannot be delegated. The agent can find candidate passages; it cannot know that §4.12 is superseded in practice by a local policy, or that two similar sections differ in a way that matters here. Everything downstream — the draft, the pin, the developer's implementation — rests on this choice.

### 3.3 Drafting

The agent fills the ticket template: summary, user story, background, acceptance criteria, use cases, and two Mermaid diagrams (a sequence diagram and a business flow). Every claim that touches a standard carries a `[doc-id §section]` citation.

### 3.4 Pinning

Once you confirm the sections, the agent calls `kb_context_new` and embeds the returned `## KB context` block. That block records the hub's commit at this moment. From now on the ticket does not say "see the handbook" — it says "this is exactly what the handbook said on the day this was written, and here is how to check."

### 3.5 Lint

`kb ticket lint` runs, the agent fixes what it reports, and it repeats until DoR: PASS.

### 3.6 Maturity review

Once lint passes, the agent runs two independent reviews against `docs/review-rubric.md` — one scoring **Business coverage**, one scoring **Dev implementability**. It applies fixes and reviews again, up to three rounds or until both axes reach 4 or higher.

A gap it cannot close itself becomes an owned open question, written `OPEN(<owner>)`, rather than a guess. The outcome lands in the ticket's `## Review record` section.

Read the open questions. They are the agent telling you where it knew it did not know.

### 3.7 Your review

The draft lands in `tickets/<ticket-id>.md`. You read it, commit it, and paste it into the tracker. The assistant never does that for you.

---

## 4. Mission plans — for large features

A feature spanning several user stories gets a **mission plan** first. Small work goes straight to a ticket. A mission is never mandatory.

```
/ba-mission-plan
```

The pipeline is **Intake** → **Ground** → **Draft** → **Split** → **Pin** → **Lint** → **Maturity review** → **Review**, and it saves `missions/M-<slug>.md`.

A mission carries:

- A **C4 Level 1** diagram (System Context) and a **C4 Level 2** diagram (Container).
- A **scope split** — what is in, what is explicitly out.
- A **user-story backlog** whose ids derive from the mission id.

```
kb mission lint missions/M-checkout-revamp.md
```

The gate enforces: required structure, both diagrams present, a well-formed backlog whose ids derive from the mission id, and every citation resolving at the pinned hub version. A `0/N US drafted` coverage warning is normal — the tickets do not exist yet.

What stays **your** judgement: whether the backlog is actually complete.

### 4.1 Linking tickets to a mission

When drafting a story from the backlog, name the parent mission. The agent then:

- reads `missions/<mission-id>.md` and takes the story title from the backlog row,
- writes `> Parent mission: <mission-id>` on its own line directly under the ticket title,
- saves the ticket as `tickets/<mission-id>-US<n>.md`.

That filename is what the back-link check looks for. Dropping the parent line is valid — the ticket just becomes untraceable to its mission.

## 4.2 Numbering

Backlog numbering gaps are fine. If you drop a story, leave its number retired. Renumbering would break the filenames of tickets already drafted.

## 5. Using code knowledge

Besides domain documents, the hub holds two documents per product repository, published by that repository's own workflow:

| Document                       | Holds                                                                      | Use it for                      |
|--------------------------------|----------------------------------------------------------------------------|---------------------------------|
| <code>&lt;repo&gt;-code</code> | generated structure — service names, technologies, dependencies, endpoints | the <i>names</i> in a diagram   |
| <code>&lt;repo&gt;-svc</code>  | curated responsibility — what each service is actually for                 | the <i>meaning</i> in a diagram |

Both appear in `kb_search` results tagged `code`, like any other hub content, so the agent can ground diagrams in them without you naming the repository specially.

Together they fill a C4 container:

```
Container(alias, label, technology, description)
  \_____/ \_____/
  <repo>-code $svc.<name> <repo>-svc $svc.<name>
```

Write `%%TODO: verify against codebase%%` only when **neither** document answers.

**One caution.** `<repo>-svc` grounds a diagram. It is never a substitute for a domain citation in an acceptance criterion. A code, format, enum or threshold that encodes a standard must still come from a pinned domain section, not from a service's responsibility text.

## 6. What the gate checks — and what it does not

### 6.1 Errors (the gate fails)

- **Required sections present:** Summary, User Story, Background, Acceptance Criteria, Use cases, both Mermaid diagrams, KB context, Definition of Ready.
- **Every `## KB context` ref resolves** at its pinned hub commit. No broken or malformed refs.
- **Every inline `[doc-id $section]` citation is backed by a pinned ref.**

A citation written in the old bare form still counts, with a warning telling you to bracket it.

## 6.2 Warnings (still your judgement)

| Warning                                                              | Why it is not an error                                                                       |
|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| An acceptance criterion touching a standard has no citation          | The tool cannot reliably tell which criteria touch a standard                                |
| The business quality of the story                                    | Not machine-checkable                                                                        |
| <code>## Review record</code> missing, empty, or still a placeholder | Lint can see the section, not whether the review was meaningful                              |
| A stale ref                                                          | The cited section was amended upstream. Still resolves, so not broken — but you should look. |
| A pinned ref the body never cites                                    | The pin may be background the ticket did not need to quote                                   |

## 6.3 Making staleness fail

```
kb ticket lint tickets/ABC-123.md --fail-on-stale
kb mission lint missions/M-checkout.md --fail-on-stale
kb resolve tickets/ABC-123.md --status-only
```

Or set the `KB_FAIL_ON_STALE` repository variable to make the CI gate do it for every ticket.

## 6.4 Why the workflow is not path-filtered

The scaffolded workflow is named `kb-ticket-lint` and triggers on every pull request, not only on changes under `tickets/` or `missions/`. That is deliberate.

GitHub never synthesises a passing status for a job that never started. A `paths:` filter on a *required* check would leave any pull request touching neither directory waiting forever for a check that will never run. Instead the job always starts, inspects the diff, and dispatches by directory — exiting cleanly with a notice when there is nothing to lint.

The workflow name also predates the mission gate. It is kept for branch-protection compatibility; renaming it would leave already-protected repositories blocked on a check that no longer exists.

# 7. When a citation goes stale

Knowledge changes. A ticket written in March against a document amended in May must not silently start meaning something else.

```
kb resolve tickets/ABC-123.md      # ok | stale | broken, per ref
kb diff hr-handbook --against <rev> # what actually changed
```

| Status | Meaning                                            | What to do                                                                             |
|--------|----------------------------------------------------|----------------------------------------------------------------------------------------|
| ok     | The pinned content still matches what is published | Nothing                                                                                |
| stale  | The source was amended after you cited it          | Run <code>kb diff</code> ; decide whether the acceptance criteria need updating        |
| broken | The reference no longer resolves at all            | Re-ground and re-pin with <code>kb context: new</code> or <code>kb_context: new</code> |

`kb diff` tells you exactly what moved: section titles, one-line summaries, the summary text, the full original, sections added and removed, and whether the document's ordering changed.

Most staleness is harmless — a typo fix upstream. Some of it is a requirement that no longer matches reality. The tool tells you which sections to look at; the judgement is yours.

## 8. Working without an assistant

Everything the agent does, you can do at a terminal.

```
kb query "refund eligibility window" --tags payments
kb get payments-spec 4.2 --level 12
kb context new --refs "payments-spec $4.2, payments-spec $4.3"
kb ticket lint tickets/ABC-123.md
kb mission lint missions/M-checkout.md
```

`kb context new` prints the block; paste it into the ticket under `## KB context`.

## 9. Troubleshooting

| Symptom                                                  | Cause                                                                                                                    | Fix                                                                                                                                                                                               |
|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Lint: "ref does not resolve"                             | The section id is wrong, or that content is not published                                                                | Re-run <code>kb query</code> and re-pin. Check the hub PR actually merged.                                                                                                                        |
| Lint: "citation not backed by a pinned ref"              | The body cites something the KB context block does not pin                                                               | Add the ref via <code>kb_context_new</code> , or remove the citation                                                                                                                              |
| Lint: "missing required heading"                         | A required section is absent — or present only inside a fenced code block                                                | Move the real heading outside the fence                                                                                                                                                           |
| Lint passes locally, fails in CI                         | <code>STRATA_KB_HUB</code> is unset in Actions variables, or the hub is private and <code>KB_HUB_TOKEN</code> is missing | Configure both; see §2.4                                                                                                                                                                          |
| The gate fails on a fork PR                              | Forks cannot read repository secrets                                                                                     | Merge through a branch in this repository                                                                                                                                                         |
| The assistant cannot reach the hub                       | <code>STRATA_KB_HUB_URL</code> or <code>STRATA_KB_HTTP_TOKEN</code> unset or wrong                                       | Re-run <code>kb mcp-setup (/kb-mcp-setup)</code> — it diagnoses which one, and a bare re-run re-verifies without retyping the token; ask the hub maintainer for a current token if it is rejected |
| <code>kb query</code> finds nothing                      | The document is not published yet, or the tags are too narrow                                                            | Drop <code>--tags</code> ; check with the hub owner that the publish PR merged                                                                                                                    |
| Everything resolves <code>broken</code> after an upgrade | The ticket carries a pin from an older block format                                                                      | Re-pin with <code>kb context new</code>                                                                                                                                                           |

## 10. Command summary

| Command                                                 | Purpose                                                                     | Exit                        |
|---------------------------------------------------------|-----------------------------------------------------------------------------|-----------------------------|
| <code>kb mcp-setup [--hub-url URL] [--no-verify]</code> | Write the hub's HTTP MCP credentials into <code>.env</code> and verify them | 0                           |
| <code>kb query &lt;text&gt; [--tags t]</code>           | Search published knowledge                                                  | 0                           |
| <code>kb get &lt;doc&gt; &lt;section&gt;</code>         | Fetch one section                                                           | 0                           |
| <code>kb tags</code>                                    | List the published tag vocabulary                                           | 0                           |
| <code>kb context new --refs "..."</code>                | Generate a pinned citation block                                            | 0 ok, 1 unresolvable ref    |
| <code>kb resolve &lt;file&gt;</code>                    | Check a ticket's refs                                                       | 0 all ok, 1 broken, 2 stale |
| <code>kb diff &lt;doc&gt; --against &lt;rev&gt;</code>  | What an amendment changed                                                   | 0                           |
| <code>kb ticket lint &lt;file&gt;</code>                | Ticket Definition-of-Ready gate                                             | 0 PASS, 1 FAIL, 2 stale     |
| <code>kb mission lint &lt;file&gt;</code>               | Mission Definition-of-Ready gate                                            | 0 PASS, 1 FAIL, 2 stale     |

Slash commands: `/ba-ticket-author`, `/ba-mission-plan`.

## 11. Upgrading the repository

```
kb init --kind ba
```

This touches **scaffold files only** — the CI workflow, the skill, command and prompt wrappers, and the templates under `docs/` — and overwrites one only when its content differs from the new template.

`.kb/config.yaml`, `.kb/index.yaml` and `.claude/settings.json` are never touched by a plain `kb init`.

**--force replaces all three outright**, unmerged: your hand-edited `.kb/config.yaml` including `kind`, `repo_id` and `asset_store`; `.kb/index.yaml`; and your entire `.claude/settings.json` including hooks, permissions and model. Use it deliberately.

Your own content under `tickets/` and `missions/` is not scaffolding. `kb init` never reads, writes or overwrites anything you author there — the only thing it places in those directories is an empty `.gitkeep`, so Git can track them before your first ticket exists.